Taller 3: Manipulación de Datos en R II

Estadística II

M. Constanza Ayala

maria.ayala@uv.cl

Agenda una reunión aquí

Escuela de Sociología - Universidad de Valparaíso

24 de marzo, 2026

Pasemos lista mientras descargan los materiales y dejamos todo listo ✨

📂 **Paso 1:** Abrimos nuestro proyecto con doble clic en el archivo `.Rproj`

- Si no tienen proyecto, creamos uno: - `File > New Project > New Directory > New Project` - Creamos carpetas: `data`, `scripts`, `figures`, `output`

💾 **Paso 2:** Movemos el archivo **"Script Taller 3 EstadisticaII.R"** a la carpeta `scripts`

📎 **Paso 3:** Verificamos que la base **ELSOC_Wide_2016_2023.RData** y **Base Evaluaciones ELPI III_reducida.RData** esté en la carpeta `data`

> **Todos los materiales los pueden descargar en este link:** Materiales clases

Objetivos de la clase

- Retomar el paquete **`dplyr` para manipulación de datos**

- **Aplicar funciones** clave: `filter()`, `select()`, `arrange()`

¿Qué vimos la semana pasada en R?

✅ Cargamos una base de datos y la exploramos
✅ Exploramos bases de datos con `glimpse()`, `head()` y `colnames()`
✅ Introdujimos `dplyr` y el operador `%>%`

💡 ¡Hoy damos paso a la **manipulación de datos** con `dplyr`!

Actividades prácticas

A lo largo de la clase realizaremos **tres actividades prácticas** para ejercitar cada función vista.

No tienen nota, pero recomiendo resolverlas en un script aparte para que quede como material de estudio.

- Crea un script nuevo (no uses el script de actividades de la clase) e incluye un título, tu nombre, correo y fecha. Por ejemplo:

```
# Actividades Taller 3 Estadística II ----------------------------------------
# M. Constanza Ayala (maria.ayala@uv.cl)
# 24-03-2026
```

De cargar a transformar datos

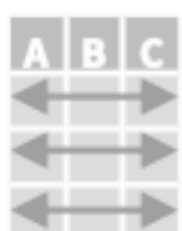
- Ya vimos cómo **importar** bases de datos en R.
- El siguiente paso es **trabajar con ellas**: explorarlas, transformarlas y limpiarlas.
- Para esto, usaremos un conjunto de funciones del paquete `dplyr`.

¿Qué es dplyr?

- Parte de **tidyverse** (colección de paquetes de R)
- Pensado para **transformar y limpiar datos**
- Permite escribir **código legible y encadenado** con %>%

dplyr functions work with pipes and expect **tidy data**. In tidy data:

&



Each **variable** is in its own **column**

Each **observation**, or **case**, is in its own **row**

pipes

x %>% f(y) becomes **f(x, y)**

Carguemos el paquete y una base de datos

```r
library(dplyr)

load("data/Base Evaluaciones ELPI III_reducida.RData")
```

Ocupemos %>%

```r
# Exploremos los datos con dplyr
data %>% glimpse() # Vista previa
```

```
Rows: 15,827
Columns: 10
$ folio          <dbl> 100011, 100081, 100091, 100121, 100131, 100151, 100161,…
$ espanel        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
$ sexo           <dbl> 2, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 2, 2, 1, 1, 2…
$ nivel_edu_niño <dbl> 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4…
$ edad_mesesr    <dbl> 144, 144, 146, 144, 143, 147, 146, 143, 143, 147, 142, …
$ idregion       <dbl> 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15,…
$ wm_pb_pa       <dbl> 33, 38, 34, 33, 41, 37, 33, 34, 32, 38, 37, 40, 23, 40,…
$ wm_pb_fd       <dbl> 48, 78, 43, 33, 80, 69, 49, 37, 60, 63, 61, 43, 31, 54,…
$ wm_pb_cc       <dbl> 18, 21, 15, 13, 25, 18, 14, 11, 21, 25, 15, 20, 4, 19, …
$ pscs_pb_sp     <dbl> 41, 37, 30, 38, 39, 37, 41, 45, 37, 28, 41, 44, 38, 44,…
```

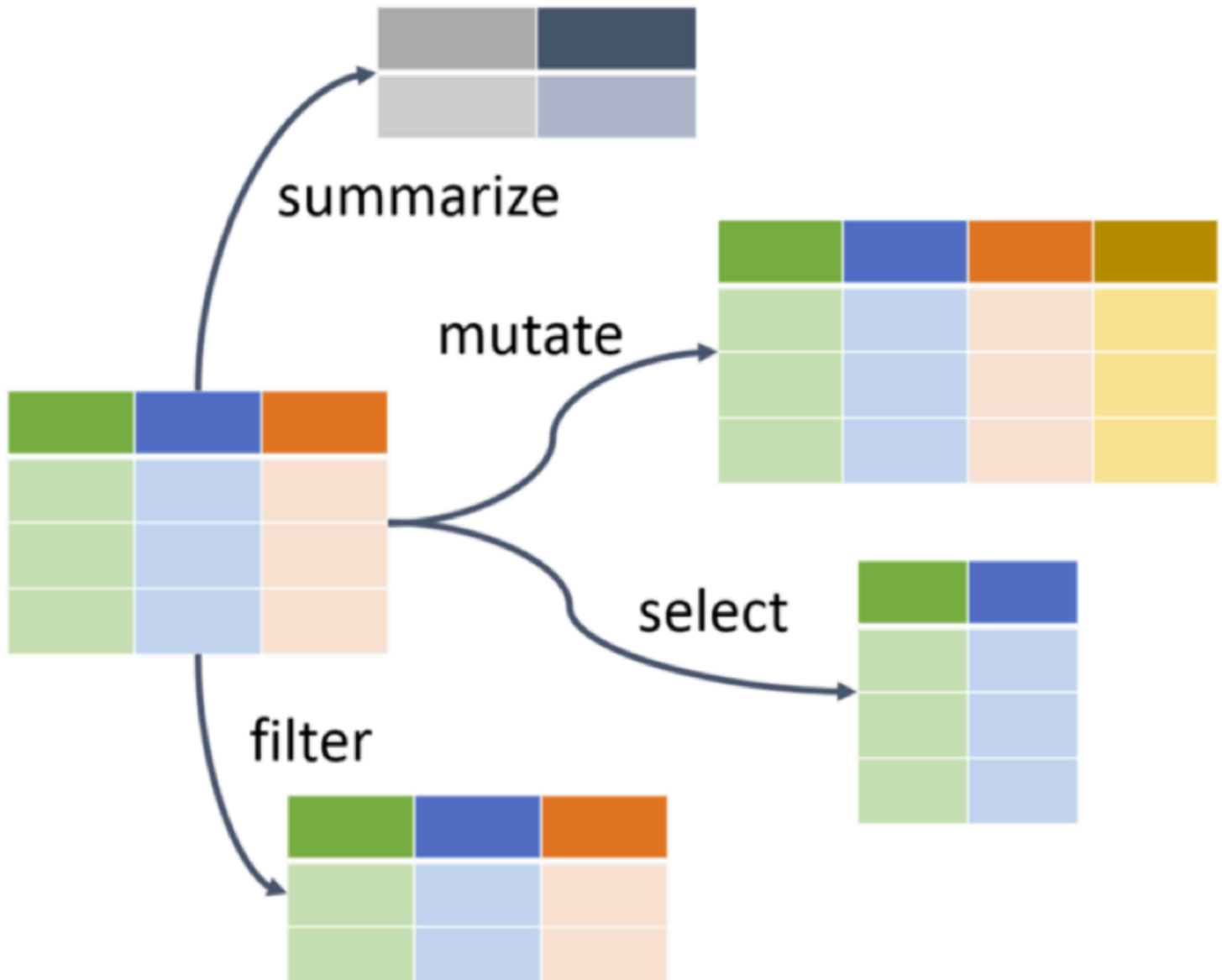
```r
data %>% head()    # Primera 6 observaciones
```

```
   folio espanel sexo nivel_edu_niño edad_mesesr idregion wm_pb_pa wm_pb_fd
1 100011       1    2              4         144       15       33       48
2 100081       1    1              4         144       15       38       78
3 100091       1    1              4         146       15       34       43
4 100121       1    1              4         144       15       33       33
5 100131       1    1              4         143       15       41       80
6 100151       1    2              4         147       15       37       69
  wm_pb_cc pscs_pb_sp
1       18         41
2       21         37
3       15         30
4       13         38
5       25         39
6       18         37
```

```r
data %>% colnames() # Nombres columnas/variables
```

```
[1] "folio"          "espanel"        "sexo"           "nivel_edu_niño"
[5] "edad_mesesr"    "idregion"       "wm_pb_pa"       "wm_pb_fd"
[9] "wm_pb_cc"       "pscs_pb_sp"
```

Manipulación de datos con dplyr



Transformación usando vectores

Row functions return a subset of rows as a new table.

filter(.data, …, .preserve = FALSE**)** Extract rows that meet logical criteria.
filter(mtcars, mpg > 20)

select(.data, …**)** Extract columns as a table.
select(mtcars, mpg, wt)

arrange(.data, …, .by_group = FALSE**)** Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
arrange(mtcars, mpg)
arrange(mtcars, desc(mpg))

filter()

- Devuelve filas que cumplen con al menos una condición lógica
- Puedes usar operadores lógicos: ==, >, <, ! =, &, |

```r
#Hombre
data_hombre <- data %>%
  filter(sexo=="Hombre")
dim(data_hombre)
```

```
[1] 8095   10
```

```r
#Mujer
data_mujer <- data %>%
  filter(sexo=="Mujer")
dim(data_mujer)
```

```
[1] 7732   10
```

- Puedes aplicar más de una condición:

```r
data_mat1 <- data %>%
  filter(wm_pb_pa >=36 & wm_pb_fd==36)
dim(data_mat1)
```

```
[1] 55 10
```

```r
data_mat2 <- data %>%
  filter(wm_pb_pa >=36 | wm_pb_fd==36)
dim(data_mat2)
```

```
[1] 3250   10
```

Operadores lógicos en R

- Sirven para hacer comparaciones y construir condiciones en funciones

Operador	Significado	Ejemplo
`==`	igual a	`species == "Human"`
`!=`	distinto de	`gender != "male"`
`>`	mayor que	`mass > 80`
`<`	menor que	`height < 180`
`&`	**y** lógico (ambas)	`species == "Human" & mass > 75`
`|`	**o** lógico (una u otra)	`species == "Human" | species == "Droid"`

Actividad práctica 1

1. Cargar el paquete `dplyr` utilizando la función `library()` en una sección del script llamada `Paquetes`.
2. Cargar la base de datos `ELSOC_Wide_2016_2023.RData` utilizando la función `load()` en una sección del script llamada `Base de datos`.
3. Explorar la base de datos utilizando el pipe de dplyr `%>%` con las funciones `glimpse()`, `head()` y `colnames()` en una sección del script llamada `Exploración base de datos`.
4. Crea una base de datos llamada `data_elsoc_filtrada` aplicando el siguiente filtro con `filter()` en una sección del script llamada `Filtro 1`:
   - Personas que hayan participado en el último levantamiento (2023) (`ola_w07`, categoría `1`)
   - Explora la nueva base de datos con `glimpse()`

select()

- Devuelve solo la o las columnas que especifiques

```r
data1 <- data %>%
  select(1:3)
data1 %>% colnames()
```

```
[1] "folio"   "espanel" "sexo"
```

```r
data2 <- data %>%
  select("folio", "wm_pb_pa")
data2 %>% colnames()
```

```
[1] "folio"    "wm_pb_pa"
```

```r
data3 <- data %>%
  select("folio", "wm_pb_pa":"wm_pb_cc")
data3 %>% colnames()
```

```
[1] "folio"    "wm_pb_pa" "wm_pb_fd" "wm_pb_cc"
```

```r
data4 <- data %>%
  select(1,edad_mesesr,sexo,idregion,wm_pb_pa:wm_pb_cc)
data4 %>% colnames()
```

```
[1] "folio"       "edad_mesesr" "sexo"        "idregion"    "wm_pb_pa"
[6] "wm_pb_fd"    "wm_pb_cc"
```

- También puedes usar: select(-columna) para excluir

```r
data5 <- data %>%
  select(-sexo)
data5 %>% colnames()
```

```
[1] "folio"          "espanel"        "nivel_edu_niño" "edad_mesesr"
[5] "idregion"       "wm_pb_pa"       "wm_pb_fd"       "wm_pb_cc"
[9] "pscs_pb_sp"
```

Actividad práctica 2

1. A partir de la base de datos `data_elsoc_filtrada`, selecciona las siguientes variables con `select()` en una sección del script llamada `Selección de variables 1`: `idencuesta`, `tipo_atricion`, `tipo_caso`. Luego, explora la base de datos con `%>%` y `glimpse()`.

2. Guarda la base de datos editada en formato RData en la carpeta `output` con el nombre `data_elsoc_filtrada`. Utiliza la función `save()`.

arrange()

- Ordena filas por una o más variables

```r
data_arrange1 <- data4 %>%
  arrange(wm_pb_pa)  # Por defecto el orden es creciente
head(data_arrange1, n=15)
```

```
    folio edad_mesesr   sexo idregion wm_pb_pa wm_pb_fd wm_pb_cc
1  102741         114 Hombre       15        0        0        0
2  124551         143 Hombre        3        0        0        0
3  127371         107 Hombre        3        0       19        0
4  135961         113  Mujer        4        0        0        0
5  141981         111 Hombre        5        0        0        0
6  166991         142  Mujer        5        0        0       NA
7  177871         117  Mujer        6        0        0        8
8  190951         129 Hombre        7        0        4       16
9  193671         145  Mujer        7        0        0        1
10 193911         142 Hombre        7        0        0        0
11 247021         121 Hombre        9        0        0        0
12 276351         140 Hombre       12        0        0        0
13 278341         149 Hombre       13        0       15        1
```

- Usa desc() para ordenar de forma descendente

```r
data_arrange2 <- data4 %>%
  arrange(desc(wm_pb_pa)) # Decreciente
head(data_arrange2, n=15)
```

```
    folio edad_mesesr   sexo idregion wm_pb_pa wm_pb_fd wm_pb_cc
1  166841         146  Mujer        5       62       90       30
2  168121         111 Hombre        5       62       60       15
3  167351         132  Mujer        5       59       90       39
4  340901         110  Mujer       13       59       70       32
5  167261         136 Hombre        5       58       80       31
6  353591         133 Hombre       13       58       85       23
7  359991         136  Mujer       13       58       50       19
8  112361         145 Hombre        4       57       69       16
9  166821         147 Hombre        5       57      120       28
10 167011         141 Hombre        5       57       70       29
11 116031         146 Hombre        2       56       37       14
12 166901         145 Hombre        5       56       67       35
13 166961         144 Hombre        5       56       53       31
```

- También puedes ordenar por varias columnas

```r
data_arrange3 <- data4 %>%
  arrange(wm_pb_pa,wm_pb_fd)  # Utilizando dos variables
head(data_arrange3, n=15)
```

```
    folio edad_mesesr   sexo idregion wm_pb_pa wm_pb_fd wm_pb_cc
1  102741         114 Hombre       15        0        0        0
2  124551         143 Hombre        3        0        0        0
3  135961         113  Mujer        4        0        0        0
4  141981         111 Hombre        5        0        0        0
5  166991         142  Mujer        5        0        0       NA
6  177871         117  Mujer        6        0        0        8
7  193671         145  Mujer        7        0        0        1
8  193911         142 Hombre        7        0        0        0
9  247021         121 Hombre        9        0        0        0
10 276351         140 Hombre       12        0        0        0
11 278571         146  Mujer       13        0        0       10
12 366311         131 Hombre       13        0        0        0
13 371491         131 Hombre       13        0        0        0
```

Actividad práctica 3

1. A partir de la base de datos `elsoc_wide_2016_2023`, crea una base de datos llamada `data_elsoc_2023` aplicando los siguientes filtros con `filter()` en una sección del script llamada `Filtros`:
   - Personas que hayan participado en el último levantamiento (2023) (`ola_w07`, categoría `1`)
   - Personas entre 20 y 60 años (`m0_edad_w07`)
   - Explora la nueva base de datos con `dim()`
2. Selecciona las siguientes variables con `select()` en una sección del script llamada `Selección de variables`: `idencuesta`, `tipo_atricion`, `tipo_caso`, `m0_sexo_w07`, `m0_edad_w07`, `comuna_w07`, `region_w07`. Luego, explora la base de datos con `glimpse()`.
3. Ordena la base de datos de forma ascendente según las variables `region_w07` e `idencuesta` usando `arrange()` en una sección del script llamada `Arrange`. El objetivo es que, dentro de cada región, los casos queden ordenados según su id.
4. Guarda la base de datos editada en formato RData en la carpeta `output` con el nombre `data_elsoc_2023`. Utiliza la función `save()`.

❓ Preguntas y aclaraciones

💬 **¿Dudas sobre lo visto hoy?**

- Tómense un momento para reflexionar y compartir preguntas sobre el contenido.
- Espacio para responder inquietudes y aclarar conceptos clave.

Resumen clase de hoy

✅ Retomamos `dplyr` para manipular datos

✅ Usamos funciones clave:

- `filter()` para seleccionar filas según condiciones
- `select()` para elegir o eliminar columnas
- `arrange()` para ordenar los datos

📅 Próxima sesión (martes)

- Continuaremos con manipulación de datos con `tidyverse`
- Ejercicio individual en R (con nota)

📚 Sugerencias lecturas para reforzar R

- Wickham & Grolemund. R for Data Science (acceso en línea biblioteca, en español: https://es.r4ds.hadley.nz/)
- AnalizaR Datos Políticos
- YaRrr! The Pirate's Guide to R